

Assignment Data Mining 2026: Detection of Fake Hotel Reviews

Instructions

This assignment should be completed by teams of three students. Register your team in Brightspace as soon as possible. Carefully read the complete assignment, including appendices, before you start working on it.

1 Introduction

Consumers increasingly review and rate products or services online. Examples of well-known review websites are TripAdvisor, Yelp and Trustpilot. With the growing popularity of these review websites, there comes an increasing potential for monetary gain through opinion spam: fake positive reviews of a company's product, or fake negative reviews of a competitor's product. Recent advancements in Natural Language Generation technology have greatly improved the quality of AI-generated reviews. In this assignment, we address the problem of detecting such AI-generated deceptive reviews. Can you tell the fake hotel reviews written by a Large Language Model (LLM) from the genuine reviews written by humans?

2 The Data

We analyze fake and genuine hotel reviews that have been collected by Ignat et al.¹ [1]. The genuine reviews have been collected from Booking.com. The fake reviews have been generated with GPT-4. We restrict our analysis to the English language reviews. There are 1000 genuine reviews and 1000 AI-generated reviews. For detailed information, read the article of Ignat et al. [1].

3 Analysis

Ignat et al. analyse the data with (Gaussian) naive Bayes and Random Forests, using word (token) frequencies and TF-IDF scores to represent the review text. They also compute Analytic Writing, Descriptiveness, and Readability scores, but you may ignore those features for this assignment. Ignat et al. also apply the more sophisticated deep learning model XLM-RoBERTa which is beyond the scope of this course.

¹The data is available from https://github.com/MichiganNLP/multilingual_reviews_deception in the file `all_data.csv`

For this assignment, we analyse the data with:

1. Multinomial naive Bayes (generative linear classifier),
2. Logistic regression with Lasso penalty (discriminative linear classifier),
3. Single classification tree (non-linear classifier),
4. Random forests (ensemble of trees), and
5. Gradient boosting (ensemble of trees).

Represent the review texts using word frequencies (bag-of-words representation). Split the data randomly into a training/validation set (75%) and a test set (25%)². Use stratified random sampling so that the class distribution in the training set and test set are the same. Use the training/validation set (1500 reviews) for training and hyperparameter selection with cross-validation or (for random forests) out-of-bag evaluation. After hyperparameter tuning, train each model on the complete training set using the selected hyperparameter values. The test set (500 reviews) is used to estimate the performance of the trained classifiers. For naive Bayes, the performance might be improved by applying some form of feature selection (in addition to removing the sparse terms). The other algorithms (trees, regularized logistic regression) have feature selection already built-in, but might still benefit from removal of sparse terms. Examples of hyperparameters are:

- λ (or $C = \frac{1}{\lambda}$) for regularized logistic regression,
- the cost-complexity pruning parameter α (called `cp` in `rpart`) for classification trees,
- the number of trees, and the number of randomly selected features for random forests,
- the shrinkage parameter (or learning rate) and tree depth in boosting,
- the value of k in selecting the top- k features for multinomial naive Bayes.

With hyperparameters we mean parameters that influence the complexity of the model, not just any parameter of the `python` or `R` function that you call to train the model. For example, the optimization algorithm used to fit a logistic regression model is not considered a hyperparameter.

You will have to make a number of choices concerning text pre-processing, feature selection, etc. You are not required to try all alternatives, but it is important that you clearly describe and motivate the choices you make. To measure predictive performance, use accuracy, precision, recall and the F_1 score.

You should address the following questions:

- (1) How does the performance of the generative linear model (multinomial naive Bayes) compare to the discriminative linear model (regularized logistic regression)?
- (2) Do the tree ensembles improve on the performance of the single tree?

²To assess the variability of results, it would be better to perform multiple random train/test splits, but for the purpose of this assignment we only perform a single split.

- (3) For the random forest classifier finally selected, is it better at distinguishing between fake and genuine *positive* reviews or between fake and genuine *negative* reviews?
- (4) For each algorithm, what are the 5 most important features in the final model? For the linear models, make a distinction between features pointing towards an AI-generated review, and features pointing towards a review written by a human. Interpret the results.

All in all, the test set should only be used to estimate the performance of five models: the selected multinomial naive Bayes, logistic regression, classification tree, random forest, and gradient boosting models. To test whether the difference in accuracy between two classifiers is significant, use McNemar’s test [2], with significance level $\alpha = 0.01$. Describe this test in your report, and give the relevant 2-by-2 table for each application of it. Note that you only have to perform a limited number of statistical tests to answer the listed questions. It is not necessary to perform all pairwise comparisons of classifier accuracy! For question (3), perform an appropriate statistical test to determine whether the difference in accuracy between positive and negative reviews is significant. For the other predictive performance measures (precision, recall and F_1 score) it is sufficient to make a qualitative comparison.

4 Data Analysis Software

You are allowed to use Python or R to perform the analysis for this assignment.

4.1 Using Python

The library `scikit-learn` contains some useful functions for feature extraction from text (see: https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction) It also contains implementations of many machine learning algorithms. For example, you can use `sklearn.linear_model.LogisticRegression` for regularized logistic regression. The penalty should be set to ‘l1’ to get Lasso regularization, but note that the regularization parameter $C = \frac{1}{\lambda}$. For tuning of the regularization parameter C , you can consider using `sklearn.linear_model.LogisticRegressionCV`. The Multinomial Naive Bayes model is implemented in `sklearn.naive_bayes.MultinomialNB`. Gradient boosting can be performed with `sklearn.ensemble.GradientBoostingClassifier`. Python also has several specialized libraries for natural language processing, such as NLTK.

4.2 Using R

We recommend the `tm` package for pre-processing the text corpus, and creating a document-term matrix. For regularized logistic regression we recommend the package `glmnet`, in particular its function `cv.glmnet` for tuning hyperparameters through cross-validation. For multinomial naive Bayes, you can use the function `multinomial_naive_bayes` from the `naivebayes` package. For classification trees you can use `rpart` and for random forests the `randomForest` package. For gradient boosting, the package `xgboost` is recommended. Read the documentation of the packages to learn about their possibilities.

5 The Report

The report should be written as a paper reporting on an empirical data mining study. This means there should be an abstract, an introduction describing and motivating the problem, a section describing the data that was used in the study, a section discussing data preprocessing, a section describing the setup of the experiments and their results, and a section in which the conclusions are presented. The experiments should be described in sufficient detail, so that the interested reader would be able to reproduce your analysis. Don't refer to the assignment in the report, as in "the assignment asked us to compare A to B, and here are the results". Write as if it was your own choice to perform this comparison, as would of course be the case if you wrote a paper about your own research.

The report must be prepared using LaTeX and the JMLR document class³, using the single-column format. The main text is limited to 10 pages, excluding the references, AI statement, and optional appendix. The optional appendix can, for example, contain tables or figures with detailed results of the hyperparameter tuning. In the main text, you can refer the reader to the appendix for additional information.

You may use any LaTeX environment you prefer. For example, Overleaf provides a browser-based LaTeX editor and is convenient if you do not want to install LaTeX locally. Alternatively, you can install a LaTeX distribution such as TeX Live (Linux/Windows), MacTeX (macOS), or MiKTeX (Windows), together with an editor such as TeXstudio or Visual Studio Code with a LaTeX extension. Whatever environment you use, make sure that the JMLR class is available and that your document compiles correctly in the required single-column format.

6 Assessment

Assessment of your work is based on the following criteria:

- Clarity, structure and lay-out (e.g. use of tables and figures) of the report.
- Methodological correctness and reproducibility of the analysis.
- Appropriateness and justification of data pre-processing choices.
- Quality of model hyperparameter tuning.
- Correct execution and interpretation of statistical tests to answer the questions.
- Sound identification and interpretation of the most predictive terms.
- Critical discussion of the limitations of your analysis.
- Correctness and completeness of the code (see code submission instructions).

Note that the assessment does not directly take into consideration the predictive performance of the selected models. This assignment is not about obtaining the highest possible performance! On the other hand, a sub par (or suspiciously good!) performance can be indicative of errors in the methodology or data pre-processing.

³<https://ctan.org/pkg/jmlr>

References

- [1] Oana Ignat, Xiaomeng Xu, and Rada Mihalcea, *MAiDE-up: Multilingual Deception Detection of AI-generated Hotel Reviews*, Findings of the Association for Computational Linguistics: NAACL 2025, pages 1636–1653.
- [2] T.G. Dietterich, *Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms*. Neural Computation, 10(7), 1895–1923, 1998.

Appendix

A Note on Using Generative AI Tools

You may use generative AI tools (e.g., ChatGPT, Claude, Copilot, Gemini) as learning aids, but not as a substitute for your own work. You are allowed to ask for explanations of concepts (e.g. what is lemmatization?), brainstorming analysis strategies, or polishing the writing of your report. It is not allowed to upload the dataset for a full solution, to submit AI-generated code or analysis as your own, or to let AI write your final report. You must understand and be able to explain every part of your analysis.

Please answer the following question in a separate AI statement at the end of the report (5-10 sentences per person is enough): Did you use any generative AI tools while working on this assignment? If yes, describe how you used them and what changes or decisions you made based on that input. (e.g., “I asked Claude how best to measure feature importance in random forests, and I followed its advice to use method ...”). If no, simply state that you did not use them. This reflection will not affect your grade directly, but failure to disclose AI use may.

B Code Submission Instructions

Along with your written report, please submit the code you used to run the experiments. Use the following format:

1. Scripts
 - (a) Submit your code as one or more `.py` (Python) or `.R` (R) files.
 - (b) Organize them clearly (e.g., `preprocessing.py`, `train_model.py`).
2. Include a `README.md` file that explains:
 - (a) How to run your code (e.g., `python main.py`).
 - (b) Which script(s) produce the key results in your report (e.g., “The performance measures reported in Table 2 are produced by `train_model.py`”).
 - (c) Which external libraries/packages are needed to run your code.
3. Reproducibility
 - (a) Your code should run start-to-finish on the provided dataset and reproduce the results you describe in your report.
 - (b) Make sure your submission is clean (remove unused code, temporary files, etc.).

C Handing in your work

Summarizing, the following items should be handed in:

1. A `.pdf` file with the report.
2. One or more `.py` or `.R` files with the code to run the experiments that you performed.
3. A `README.md` file explaining how to run your code.

These files should be collected in a `.zip` file, which should be handed in through Brightspace.